King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 13
Graphs 2

Term 252

Reading Assignment

- "Data Structures and Algorithms in Java", 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-981-4392-78-5
  - Chapter 8
    - Section 8.3: Shortest Paths (8.3.1 is not included)
    - Section 8.4: Cycle Detection (8.4.1 is not included)
    - Section 8.5: Spanning Trees
    - Section 8.6: Connectivity

Greedy Algorithms: Introduction

- An optimization problem is one that has multiple solutions, each having a specific cost.
  - Our objective is to find the best of all possible solutions.

- Greedy algorithms are typically used to solve optimization problems.

- A greedy algorithm works in phases. At each phase:
  - Make the choice that looks best at the moment regardless future consequences.
  - Hope that by choosing a local optimum at each phase, global optimum can be achieved.
  - Once a choice is made it cannot be un-done on subsequent steps of the algorithm.

- Advantages: Simplicity and Efficiency

- Greedy algorithms are simple and appealing, but they don't always give the best solution.

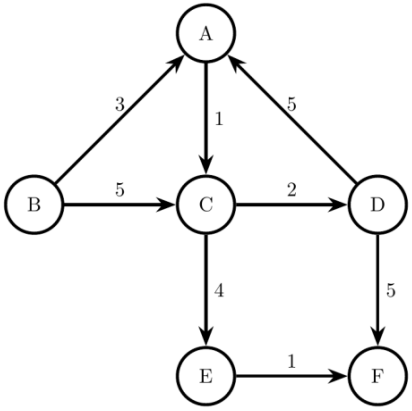
  However, greedy algorithms can often come close to the globally optimal solutions.

- Some Greedy Algorithms in Graphs
  - Shortest Path: Dijkstra's Algorithm for single source
  - Minimum Spanning Trees: Prim's Algorithm and Kruskal' Algorithm

What is the shortest path problem?

- Given a weighted graph, what is the minimum-cost path between two vertices?
  - The weight of an edge measures the cost of traveling that edge.
  - For example, in a graph representing a network of airports, the weights could represent distance, cost or time.

- Such a graph could be used to answer any of the following
  - To get from A to B, What is the
    - fastest way
    - least expensive
    - shortest distance



- Each of these questions is an instance of the same problem:

The shortest path problem!

The Dijkstra's Algorithm

For each vertex, the algorithm keeps track of its current distance from the starting vertex and the predecessor on the current path

- Goal: Given a weighted digraph $G = (V, E)$ and a source vertex S, find the shortest distance from the source vertex to every vertex in the digraph:

$$\text{dist}(v) = \text{minimum cost from } S \text{ to } v \; \forall \; v \in V$$

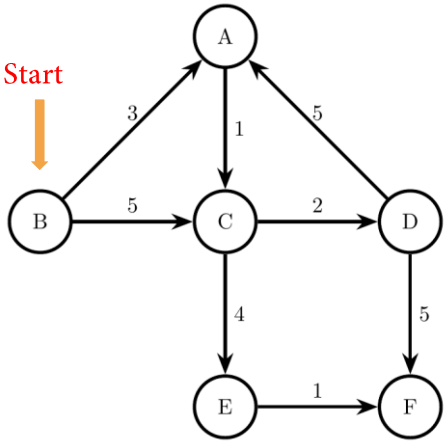
- Dijkstra algorithm computes the shortest path from a single source vertex to all other vertices in a graph with Non-negative edge weights only for directed or undirected graphs

- How it works? It repeats a two-step process
  - Choose the next vertex to explore
    - Unvisited vertex with the smallest cost
  - Update estimates:
    - By looking to the vertices connected to the current vertex

- How can we implement Dijkstra's algorithm?
- What is the time complexity of the algorithm?

Example

Important Observation: Any subpath of a shortest path must also be a shortest path.



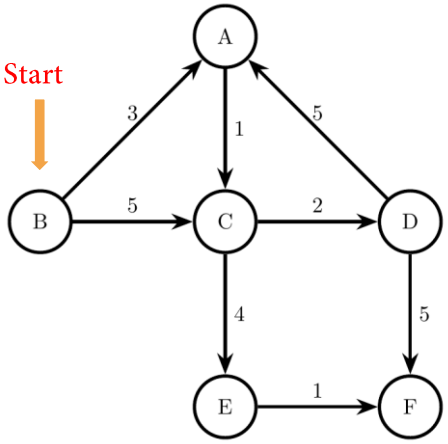
- Tracing Dijkstra's algorithm starting at vertex B:
  - Start vertex has distance 0
  - All other vertices have distance ∞
  - Repeat until all vertices are visited
    - Pick closest unknown vertex *u*. i.e. unvisited vertex with the smallest cost
    - Add it to the set of visited vertices
    - Update "best-so-far" distances for vertices with edges from u: d(v) = min(d(u)+w(u,v) , d(v))

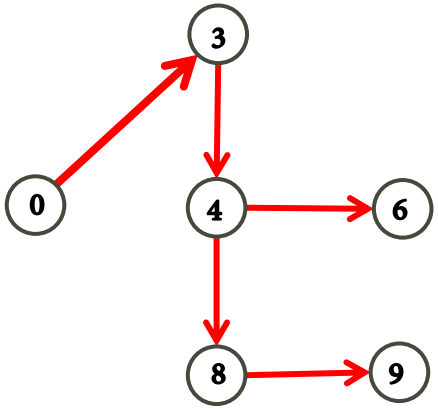
		Step							
Pass	initially	1	2	3	4	5	6	distance	Predecessor
Active Vertex		B							
A	∞								
B	0								
C	∞								
D	∞								
E	∞								
F	∞								

d(v) = min(d(u)+w(u,v) , d(v))

Example

- Once a vertex is marked known, its shortest path is known
- While a vertex is not known, another shorter path might be found





The resulting vertex-weighted graph is:

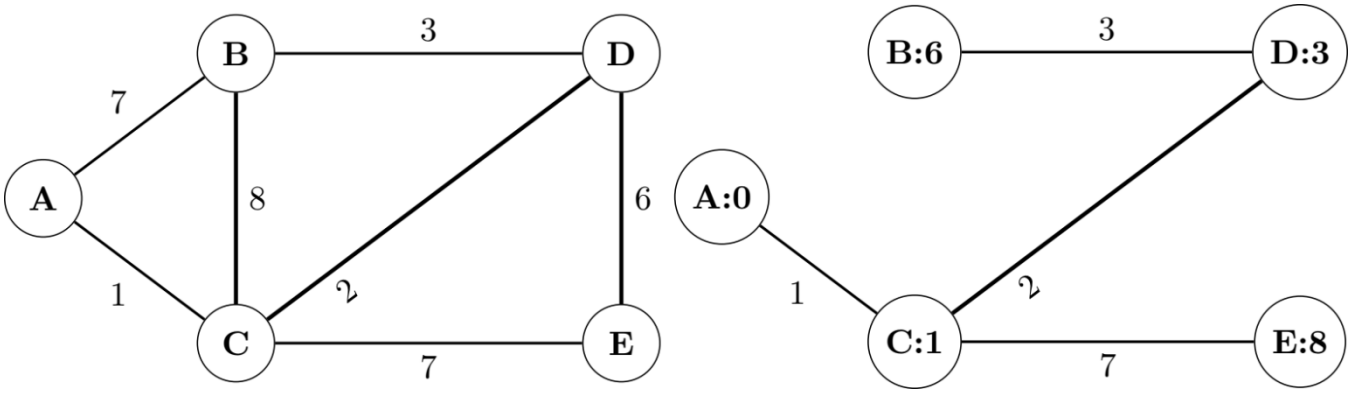
$d(v) = \min(d(u)+w(u,v) , d(v))$

- Tracing Dijkstra's algorithm starting at vertex B:
  - Start vertex has distance 0
  - All other vertices have distance ∞
  - Repeat until all vertices are visited
    - Pick closest unknown vertex *u*. i.e. unvisited vertex with the smallest cost
    - Add it to the set of visited vertices
    - Update "best-so-far" distances for vertices with edges from u:

      $d(v) = \min(d(u)+w(u,v) , d(v))$

		Step							
Pass	initially	1	2	3	4	5	6	distance	Predecessor
Active Vertex		B 0	A 3	C 4	D 6	E 8	F 9		
A	∞	3						3	B
B	0							0	-
C	∞	5	4					4	A
D	∞	∞	∞	6				6	C
E	∞	∞	∞	8	8			8	C
F	∞	∞	∞	∞	11	9		9	E

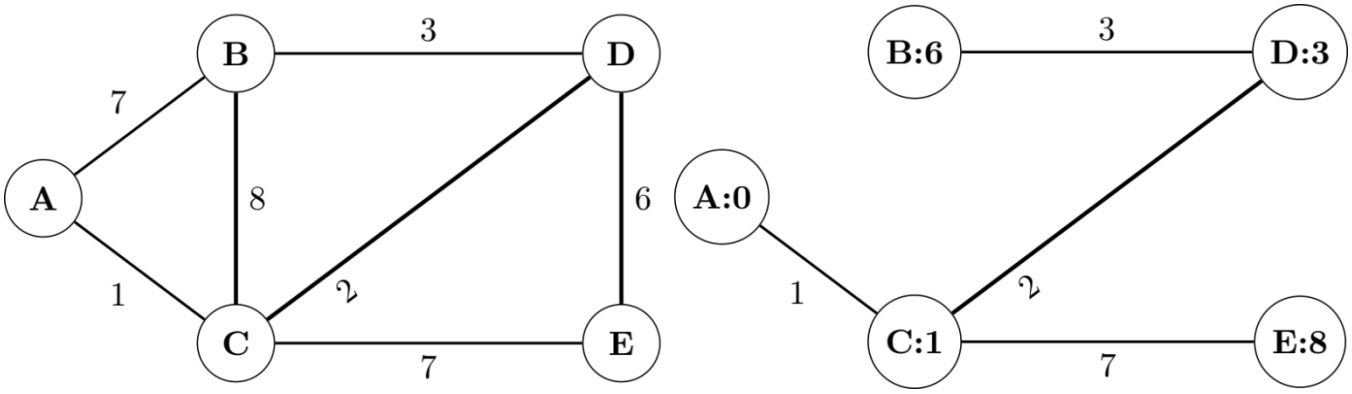
Trace Dijkstra's algorithm starting at vertex A:



- Start vertex has distance 0
- All other vertices have distance ∞
- Repeat until all vertices are visited
  - Pick closest unknown vertex u i.e. unvisited vertex with the smallest cost
  - Add it to the set of visited vertices
  - Update "best-so-far" distances for vertices with edges from u: d(v) = min(d(u)+w(u,v) , d(v))

		Step							
Pass	initially	1	2	3	4	5	6	distance	Predecessor
Active Vertex		A 0							
A	**0**								
B	∞								
C	∞								
D	∞								
E	∞								

Trace Dijkstra's algorithm starting at vertex A:



- Start vertex has distance 0
- All other vertices have distance ∞
- Repeat until all vertices are visited
  - Pick closest unknown vertex u. i.e. unvisited vertex with the smallest cost
  - Add it to the set of visited vertices
  - Update "best-so-far" distances for vertices with edges from u: d(v) = min(d(u)+w(u,v) , d(v))

		Step							
Pass	initially	1	2	3	4	5	6	distance	Predecessor
Active Vertex		A 0	C 1	D 3	B 6	E 8			
A	**0**							0	-
B	∞	7	7	6				6	D
C	∞	1						1	A
D	∞	∞	3					3	C
E	∞	∞	8	8	8			8	C

The Dijkstra's Algorithm

```
int n = graph.size();
int[] dist = new int[n];

Arrays.fill(dist, Integer.MAX_VALUE);
Arrays.fill(parent, '-');        // no predecessor initially
dist[start] = 0;

// Min-heap storing (distance, node)
PriorityQueue<Node> pq = new PriorityQueue<>(Comparator.comparingInt(node -> node.dist));

pq.add(new Node(start, 0));
boolean[] visited = new boolean[n];

while (!pq.isEmpty()) {
    Node curr = pq.poll();
    int u = curr.node;
    int currDist = curr.dist;

    if (visited[u]) continue;
    visited[u] = true;

    for (Edge e : graph.get(u)) {
        int v = e.to;
        int w = e.weight;

        if (currDist + w < dist[v]) {
            dist[v] = currDist + w;
            parent[v] = vertics[u];          // **
            pq.add(new Node(v, dist[v]));
        }
    }
}

        }
```

```
class Node {
    int node;
    int dist;

    Node(int node, int dist) {
        this.node = node;
        this.dist = dist;
    }
}
```

```
class Edge {
    int to, weight;
    Edge(int to, int weight) {
        this.to = to;
        this.weight = weight;
    }
}
```

Input: A weighted digraph $G=(V,E)$, a source vertex s

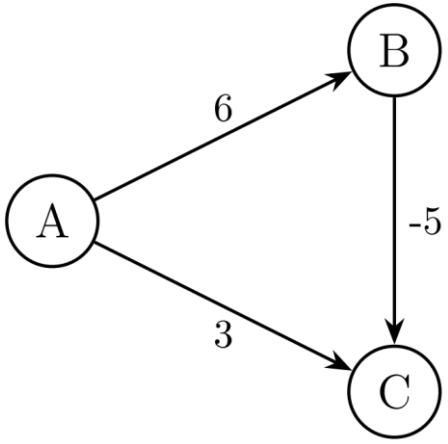
```
// dist[v] = best-known distance from the s to vertex v
for(each vertex v){
   dist[v] = ∞;    predecessor[v] = undefined;
}
Dist[s] = 0;
T = V;
while(T ≠ ϕ){
   v = a vertex in T with minimal distance from s;
   T = T – {v};
   for(each vertex u adjacent to v and in T){
     if(  dist[v] + weight(v,u) < dist[u]{
             dist[u] = dist[v] + weight(v,u);
      predecessor[u] = v;
     }
   }
}
```

Adjacency Matrix	Min heap
$O(\|v\|^2)$	$O((\|E\| + \|V\|) \log \|V\|)$

https://www.cs.usfca.edu/~galles/visualization/Dijkstra.html

When Doesn't Dijkstra's Work?

- Starting at A, which vertex do we add first?
  - C, using edge 3, because 3 < 6
- Is 3 the correct shortest path cost to C?
  - No! Going through A, we could have gotten a path of length 1

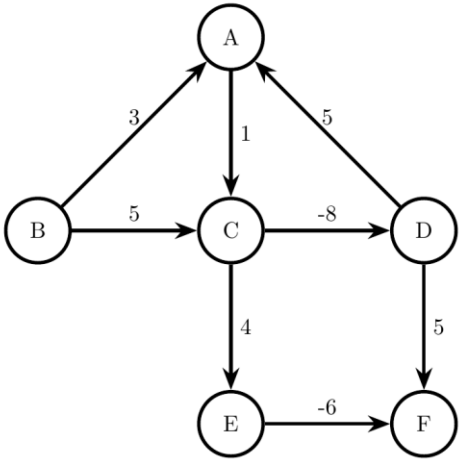


- Dijkstra's Algorithm is not guaranteed to work with negative edge weights
  - It can work but is fooled if a negative edge hides behind a large edge weight.
  - It will still run, but give wrong answer
  - Use Bellman–Ford algorithm;
- Dijkstra algorithm works with Non-negative edge weights only for directed or undirected graphs

- The problem is well-defined if a finite minimum path cost exists.
  - All edge weights are non-negative or
  - Negative edge weights with no cycle

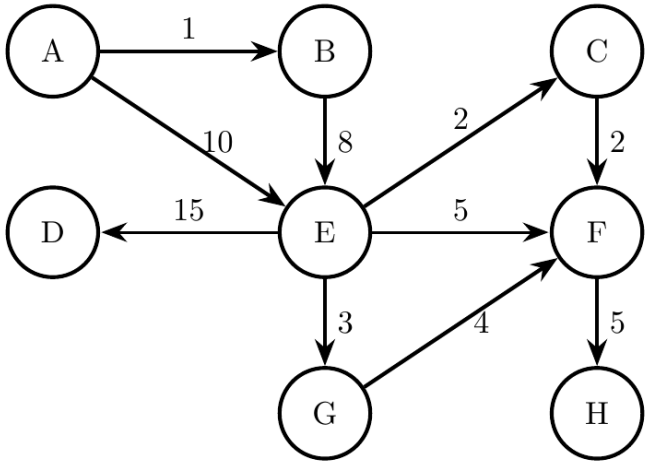
- If the graph has a negative-weight cycle, the problem is not well defined

- Shortest paths can be computed for graphs with negative edges (Total path cost stays bounded), but not for graphs with negative cycles (There is no minimal cost)



Review Questions

- Use the graph Gc shown below to trace the execution of Dijkstra's algorithm as it solves the shortest path problem starting from vertex a.
- Dijkstra's algorithm works as long as there are no negative edge weights. Given a graph that contains negative edge weights, we might be tempted to eliminate the negative weights by adding a constant weight to all the edges. Explain why this does not work.
- Dijkstra's algorithm can be modified to deal with negative edge weights (but not negative cost cycles). Implement this modified algorithm.



Minimum Spanning Tree

- What is a Minimum Spanning Tree
- Constructing Minimum Spanning Trees
- What is a Minimum-Cost Spanning Tree
- Applications of Minimum Cost Spanning Trees
- Prim's Algorithm
  - Example
  - Implementation
- Kruskal's algorithm
  - Example
  - Implementation
- Review Questions

- Some applications of the Minimum-cost spanning:
  - Building cable networks that join *n* locations with minimum cost.
  - Building a road network that joins n cities with minimum cost.
  - Obtaining an independent set of circuit equations for an electrical network.
  - In pattern recognition minimal spanning trees can be used to find noisy pixels.

What is a Spanning Tree.

- Goal: Let G = (V, E) be a simple, connected, undirected graph that is not edge-weighted. Find a minimal subset of edges such that G is still connected

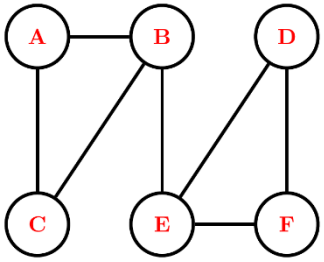
- A spanning tree of G is a free tree (i.e., a tree with no root) with |V| - 1 edges that connects all the vertices of the graph.
  - any vertex may be chosen to serve as the root of the tree.

- Thus, a spanning tree for G is a graph, T = (V', E') with the following properties:
  - V' = V ; it includes all the vertices of G
  - T is connected ; there is a path from any vertex to any vertex
  - T is acyclic ; no cycles
  - |E| = |v|-1

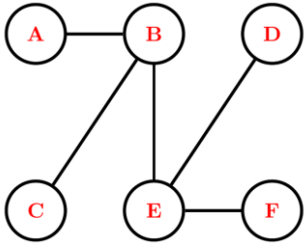
- A spanning tree is called a tree because every acyclic undirected graph can be viewed as a general, unordered tree.

Constructing Spanning Trees: Two Approaches

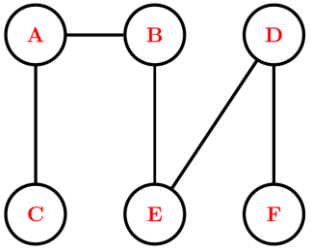
- (1) Do a graph traversal:
  - Any traversal of a connected undirected graph visits all the vertices in that graph.
  - The set of edges which are traversed during a traversal forms a spanning tree.
  - For example, Fig:(b) shows the spanning tree obtained from a breadth-first traversal starting at vertex b.
  - Similarly, Fig:(c) shows the spanning tree obtained from a depth-first traversal starting at vertex c.



(a) Graph G



(b) Breadth-first spanning tree of G rooted at b

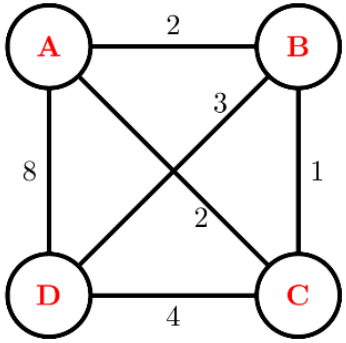


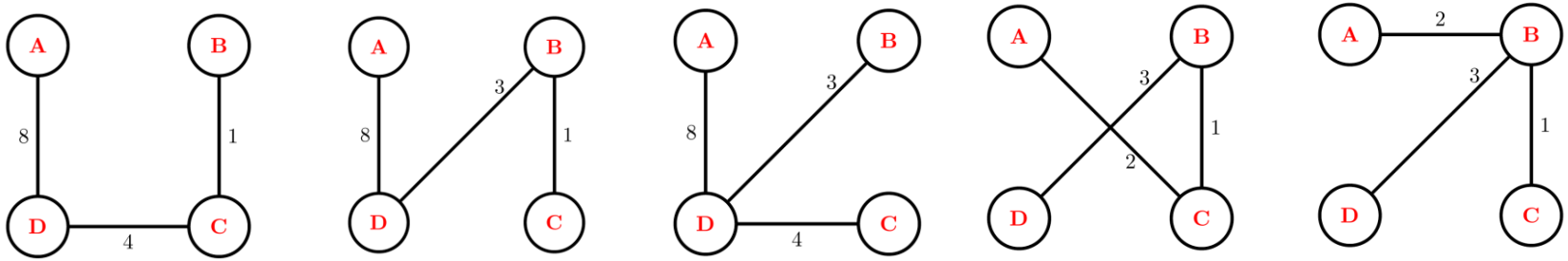
(c) Depth-first spanning tree of G rooted at c

- (2) Add acyclic edges

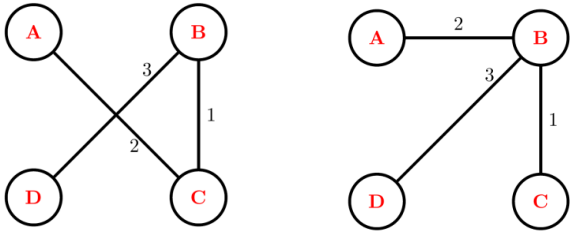
What is a Minimum-Cost Spanning Tree

- For an edge-weighted , connected, undirected graph, G, the total cost of G is the sum of the weights on all its edges.
- A minimum-cost spanning tree for G is a spanning tree of G that has the least total cost.
- Example: The graph has 16 spanning trees. Some are:
- Applications:
  - Building cable/road networks that join n locations with minimum cost.





The graph has two minimum-cost spanning trees, each with a cost of 6:



Prim's Algorithm

- Prim's algorithm finds a minimum cost spanning tree by selecting vertices from the graph one-by-one as follows:
- It starts with a tree, T, consisting of the starting vertex, s.
- At each step, it selects the minimum-weight edge that connects a vertex in T to a vertex u outside of T. The vertex u is added to T.
- This process repeats until all vertices are included.

```
Input: A graph G = (V, E) and Start vertex s
Let T set of vertices included in the growing MST
Start with T = {s}
While |T| != |V|{ // until all vertices included in T:
        v = a vertex not in T with minimal distance from any
        vertex in T ;
         Add the new vertex v to T
}
```

Input: A weighted digraph $G=(V,E)$, a source vertex s

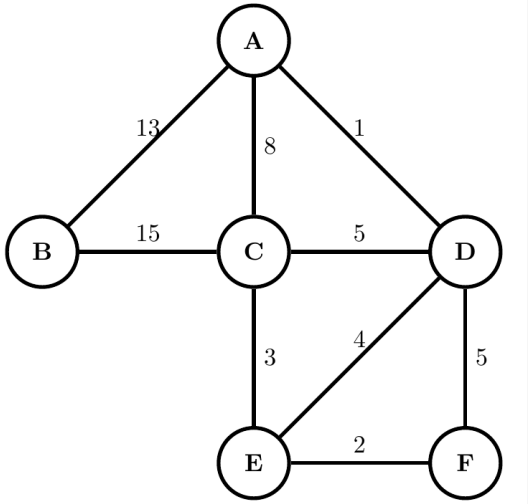
```
// dist[v] = best-known distance from the s to vertex v
Dist[s] = 0;
for(each vertex v){
  dist[v] = ∞;    predecessor[v] = undefined;
}
T = V;
while(T ≠ ϕ){
  v = a vertex in T with minimal distance from s;
  T = T – {v};
  for(each vertex u adjacent to v and in T){
   if( weight(v,u) < dist[u] {
          dist[u] =  weight(v,u);
    predecessor[u] = v;
   }
  }
}
```

- How can we implement Prim's algorithm?
- What is the time complexity of the algorithm?
  - LIKE Dijkstra's algorithm

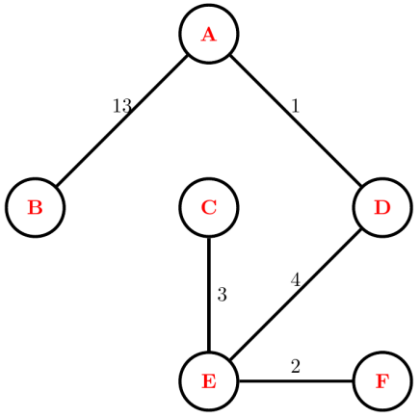
Think of growing a tree starting from any vertex.
Add the cheapest edge that connects a vertex inside the tree to a vertex outside the tree.

Example

- Trace Prim's algorithm starting at vertex A:



The resulting minimum-cost spanning tree is:

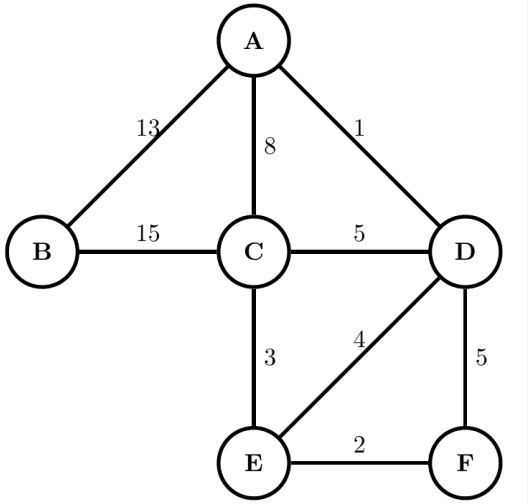


Pass	Initially	1	2	3	4	5	6	weight	Predecessor
Active Vertex		a							
a	0								
b	∞								
c	∞								
d	∞								
e	∞								
f	∞								

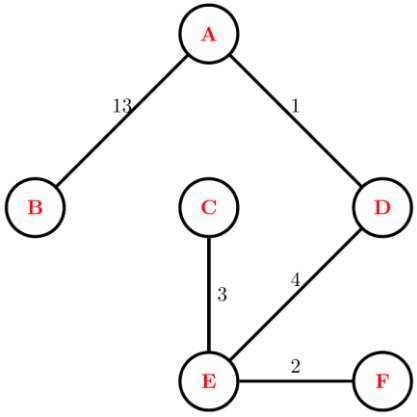
Example

Perform Prim's algorithm starting from B

- Trace Prim's algorithm starting at vertex A:



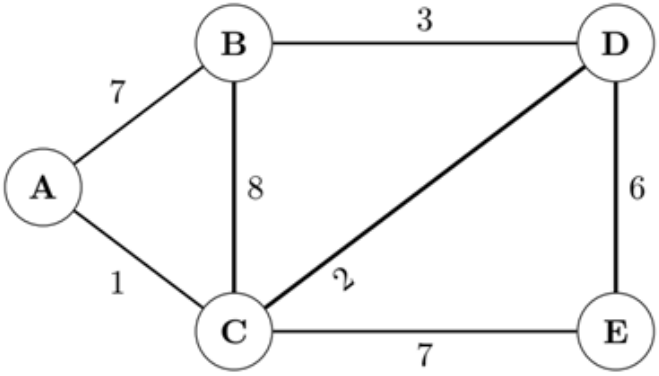
The resulting minimum-cost spanning tree is:

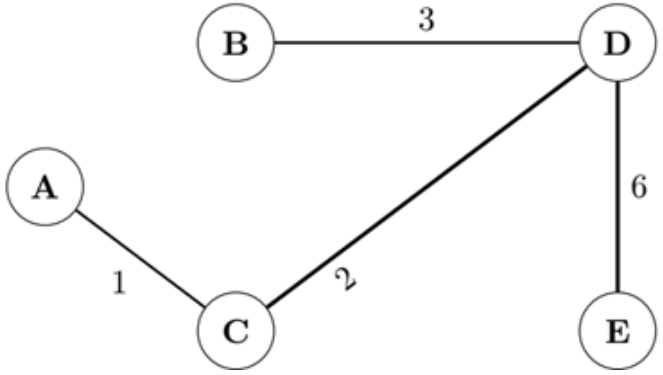


Pass	Initially	1	2	3	4	5	6	weight	Predecessor
Active Vertex		a	d	e	f	c	b		
a	0							0	-
b	∞	13	13	13	13	13		13	a
c	∞	8	5	3	3			3	e
d	∞	1						1	a
e	∞	∞	4					4	d
f	∞	∞	5	2				2	e

https://opendsa-server.cs.vt.edu/embed/PrimPE

Trace Prim's algorithm starting at vertex A:

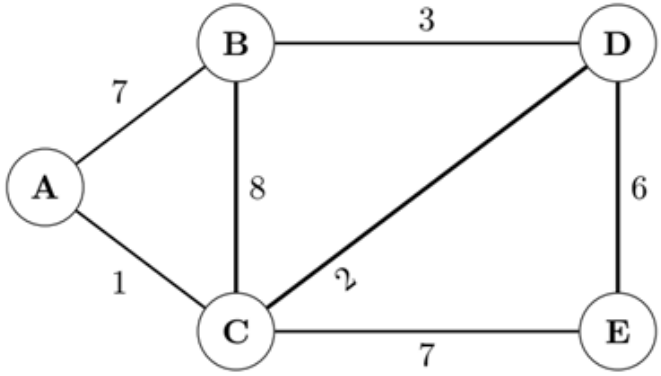


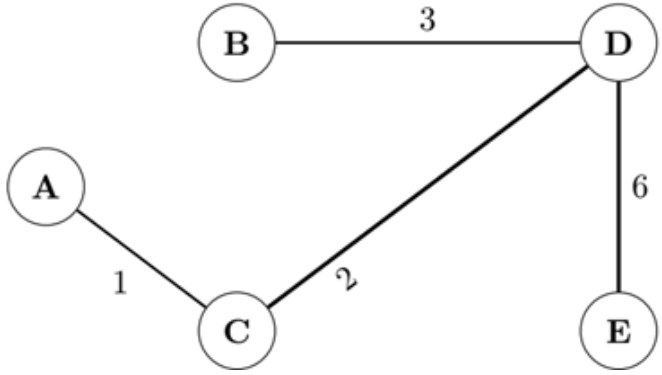


- Start vertex has distance 0
- All other vertices have distance ∞
- Repeat until all vertices are visited
  - Pick closest unknown vertex u i.e. unvisited vertex with the smallest cost
  - Add it to the set of visited vertices
  - Update "best-so-far" distances for vertices with edges from v:
    d(v) = min(w(u,v) , d(v))

		Step							
Pass	initially	1	2	3	4	5	6	distance	Predecessor
Active Vertex		A	C	D	B	E			
A	**0**								
B	∞								
C	∞								
D	∞								
E	∞								

Trace Prim's algorithm starting at vertex A:





- Start vertex has distance 0
- All other vertices have distance ∞
- Repeat until all vertices are visited
  - Pick closest unknown vertex u. i.e. unvisited vertex with the smallest cost
  - Add it to the set of visited vertices
  - Update "best-so-far" distances for vertices with edges from v:
    d(v) = min(w(u,v) , d(v))

		Step							
Pass	initially	1	2	3	4	5	6	distance	Predecessor
Active Vertex		A	C	D	B	E			
A	**0**							0	-
B	∞	**7**	7	3				3	D
C	∞	**1**						1	A
D	∞	∞	2					2	C
E	∞	∞	7	6	6			6	D

Kruskal's Algorithm.

- Kruskal's algorithm also finds the minimum cost spanning tree of a graph by adding edges one-by-one.

```
enqueue edges of G in a queue in increasing
order of cost.
T = ϕ ;
while(queue is not empty){
   dequeue an edge e;
   if(e does not create a cycle with edges in T)
      add e to T;
}
return T;
```

Kruskal's algorithm starts by sorting all edges in increasing order of weight.

It then grows the MST edge by edge:

- Take the cheapest edge in the whole graph.
- Add it if it does not create a cycle
- Move to the next cheapest edge.
- Continue until you have $|V|-1|$ edges.

How can we implement Kruskal's algorithm?

What is the time complexity of the algorithm?

MinHeap or sorting: O(|E| log |E|)

Kruskal's Algorithm In Java

```java
// Kruskal using sorted list of edges
public static List<Edge> kruskal(int n, List<Edge> edges) {

    // Sort edges by weight (ascending)
    edges.sort(Comparator.comparingInt(e -> e.weight));

    UnionFind uf = new UnionFind(n);
    List<Edge> mst = new ArrayList<>();

    for (Edge e : edges) {
        if (uf.union(e.u, e.v)) {  // If adding edge does NO
            mst.add(e);
            if (mst.size() == n - 1) break;  // MST complete
        }
    }

    return mst;
}
```

```java
// Kruskal using a Min-Heap
public static List<Edge> kruskal(int n, List<Edge> edges) {

    // Min-Heap: smallest weight removed first
    PriorityQueue<Edge> pq =
            new PriorityQueue<>(Comparator.comparingInt(e -> e.weight));
    pq.addAll(edges);

    UnionFind uf = new UnionFind(n);

    List<Edge> mst = new ArrayList<>();

    while (!pq.isEmpty() && mst.size() < n - 1) {
        Edge e = pq.poll();  // get smallest edge

        // If adding edge does NOT cause cycle
        if (uf.union(e.u, e.v)) {
            mst.add(e);
        }
    }

    return mst;
}
```

```java
class Edge {
    int u, v, weight;
    Edge(int u, int v, int weight) {
        this.u = u;
        this.v = v;
        this.weight = weight;
    }
}
```

```java
public static void main(String[] args) {

    int n = 6; // number of vertices 0..5

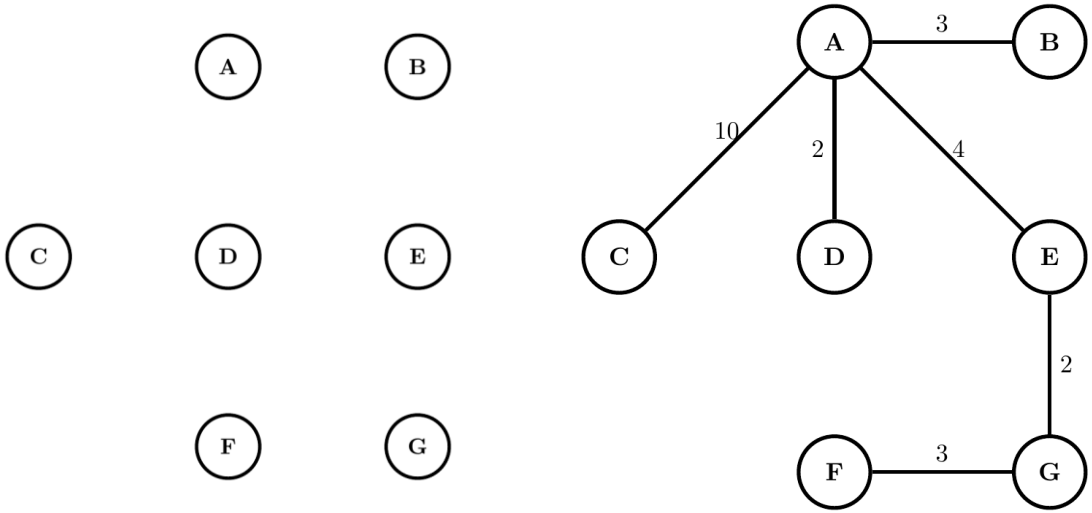
    List<Edge> edges = Arrays.asList(
            new Edge(0, 1, 4),
            new Edge(0, 2, 1),
            new Edge(2, 1, 2),
            new Edge(1, 3, 1),
            new Edge(2, 3, 5),
            new Edge(3, 4, 3),
            new Edge(4, 5, 2)
    );

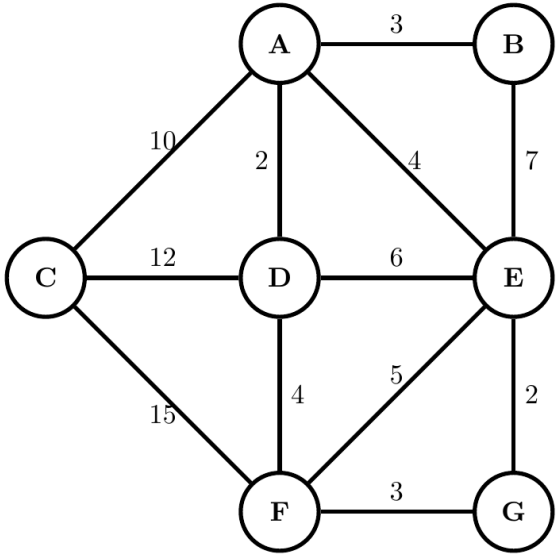
    List<Edge> mst = kruskal(n, edges);
```

```java
    System.out.println("Edges in MST:");
    int total = 0;
    for (Edge e : mst) {
        System.out.println(e.u + " -- " + e.v + " (w=" + e.weight + ")");
        total += e.weight;
    }
    System.out.println("Total MST Cost = " + total);
```

Example for Kruskal's Algorithm.

- Trace Kruskal's algorithm in finding a minimum-cost spanning tree for the undirected, weighted graph given below:



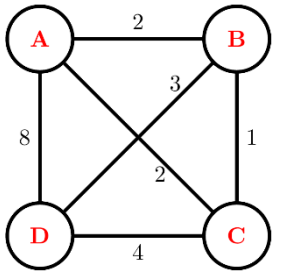


edge	AD	EG	AB	FG	AE	DF	EF	DE	BE	AC	CD	CF
weight	2	2	3	3	4	4	5	6	7	10	12	15
Insertion status	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓	✗	✗
Insertion order	1	2	3	4	5					6		

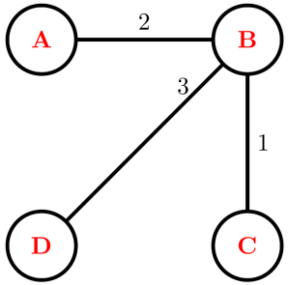
The minimum cost is: 24

Prim's and Kruskal's Algorithms

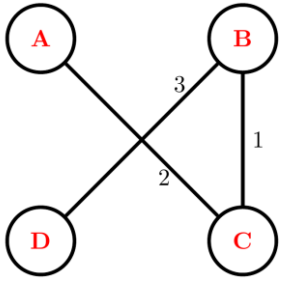
- Note: It is not necessary that Prim's and Kruskal's algorithm generate the same minimum-cost spanning tree.



- For example, for the graph:
  - Kruskal's algorithm (that imposes an ordering on edges with equal weights) results in the following minimum cost spanning tree:



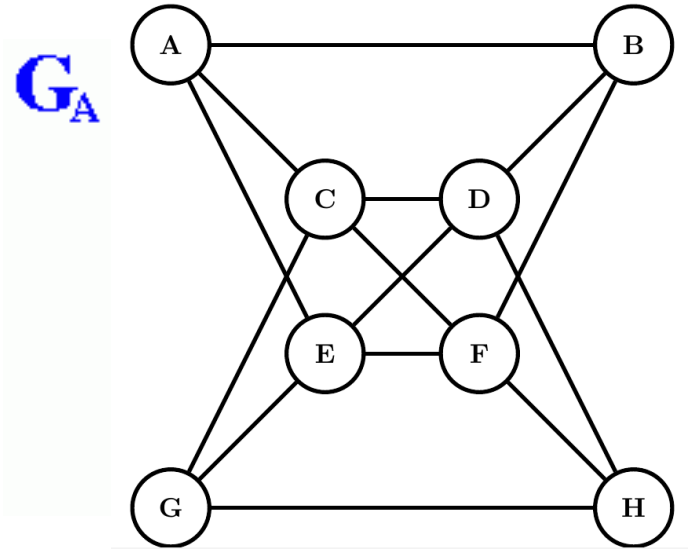
  - The same tree is generated by Prim's algorithm if the start vertex is any of: A, B, or D; however, if the start vertex is C the minimum cost spanning tree is:

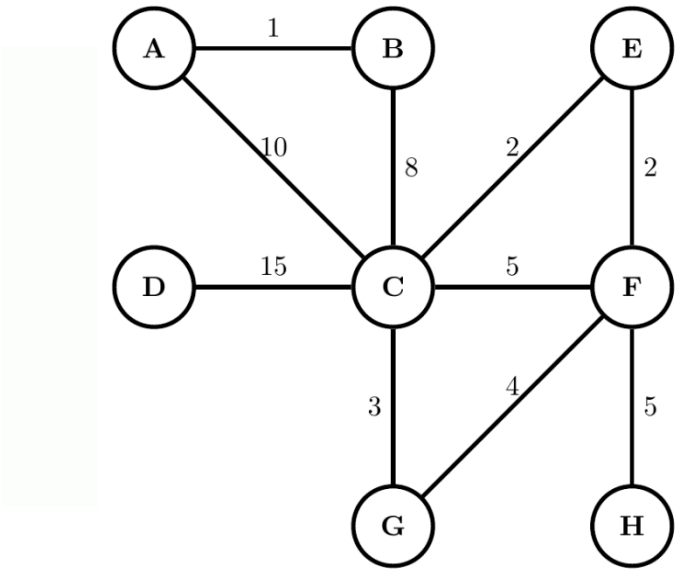


Prim's and Kruskal's Algorithms

Prim's Algorithm	Kruskal's Algorithm
It starts from any vertex	It starts from the smallest edge
The tree that we are making or growing always remains connected	The tree that we are making or growing usually remains disconnected
It grows a solution starting from a random vertex by repeatedly adding the vertex connected to the existing tree by the cheapest edge.	It grows a solution from the cheapest edge by adding the next cheapest edge to the existing tree
It works best on dense graphs	It works best on sparse graphs

Review Questions





- Find the breadth-first spanning tree and depth-first spanning tree of the graph GA shown above.
- For the graph GB shown above, trace the execution of Prim's algorithm as it finds the minimum-cost spanning tree of the graph starting from vertex a.
- Repeat question 2 above using Kruskal's algorithm.

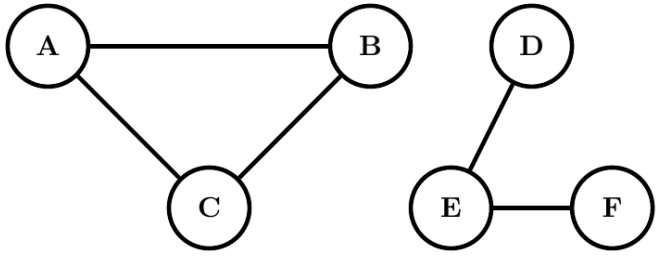
Graph Connectedness and Cycles

- Connectedness of an Undirected Graph
  - Checking Connectedness
  - Finding connected components
- Connectedness of a Directed Graph
  - Checking Connectedness
  - Finding strongly connected components
- Cycle Detection
  - Cycle Detection in Undirected Graphs
  - Cycle detection in Directed Graphs
- Review Questions.

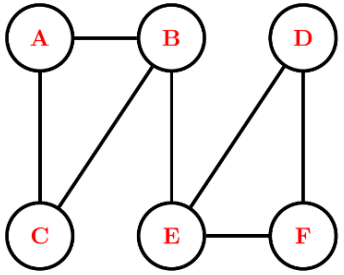
Connectedness of an Undirected Graph

- Connected Graph: An undirected graph G = (V, E) is connected if there is a path between every pair of vertices. You can reach any vertex from any other vertex. G2
- Disconnected Graph: If at least one vertex cannot be reached from some other vertex.
  - Although the figure below appears to be two graphs, it is a single graph.
  - Clearly, G1 is not connected. e.g. no path between A and D.
- Connected Components in an Undirected Graph
  - G1 consists of two unconnected parts, each of which is a connected component.
  - A connected component of an undirected graph is a maximal set of vertices such that every pair of vertices in the set is connected by a path. No connection between components.

G1 = (V, E)



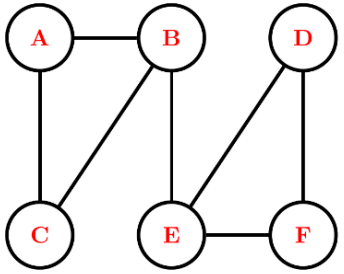
Component 1 {A, B, C} and Component 2 {E, D, F}



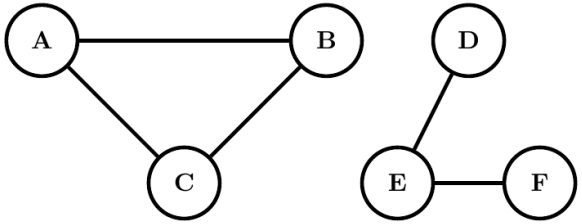
G2 = (V, E)

Algorithm for Undirected Graphs Connectedness

- Checking Connectedness
  - Can be done using DFS (Depth-First Search) or BFS (Breadth-First Search):
  - Start from any vertex v.
  - Mark all vertices reachable from v using DFS/BFS.
  - If all vertices are visited, the graph is connected.
  - Otherwise, it is disconnected.



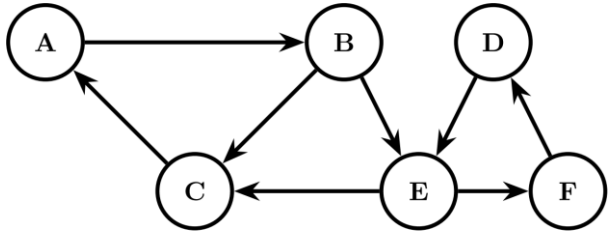
- Applications:
  - Network design: ensuring every computer can communicate
- Finding Connected Components
  - Can be done using DFS or BFS. One method call for DFS **(i.e., not General Traversal Algorithm)**
  - Initialize visited[] = false for all vertices.
  - Initialize componentCount = 0.
  - For each vertex v:
  - If visited[v] == false:
  - Do DFS starting from v.
  - Increment componentCount.
  - Return componentCount and the vertices in each component.



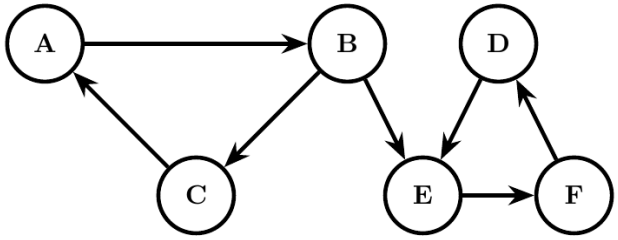
Connectedness of a Directed Graph

- Strongly connected Graphs: a directed graph G = (V, E) is strongly connected if there is a directed path between every pair of vertices.
  - G1 is Strongly connected Graph

G1 = (V, E)



- Weakly connected Graphs:
  - A directed graph G =(V, E) is weakly connected if:
  - The underlying undirected graph is connected, and if
  - There is at least one pair of vertices v1 and v2 that are not reachable from one another in the original graph.
  - G2 is a weakly connected directed graph



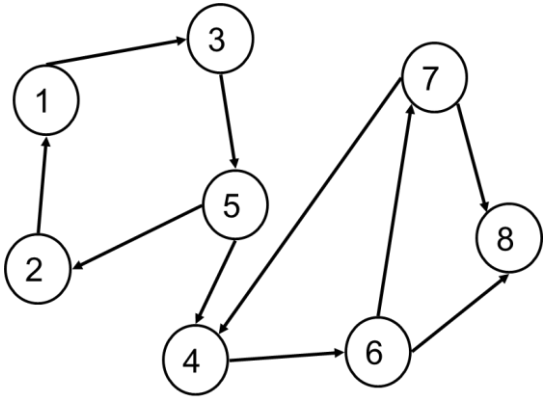
G2 = (V, E)

- Checking Strong Connectedness
  - Use |V| traversals
  - The graph is strongly connected if all the vertices are visited in each traversal.
  - What is the time complexity of this algorithm? $O(|V| * (|V| + |E|)) = O(|V|^2 + (|V||E|))$

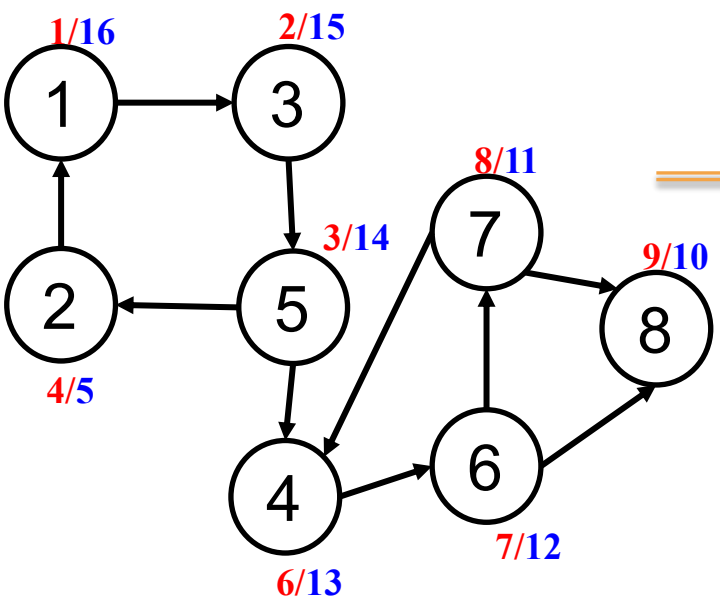
Strongly Connected Components

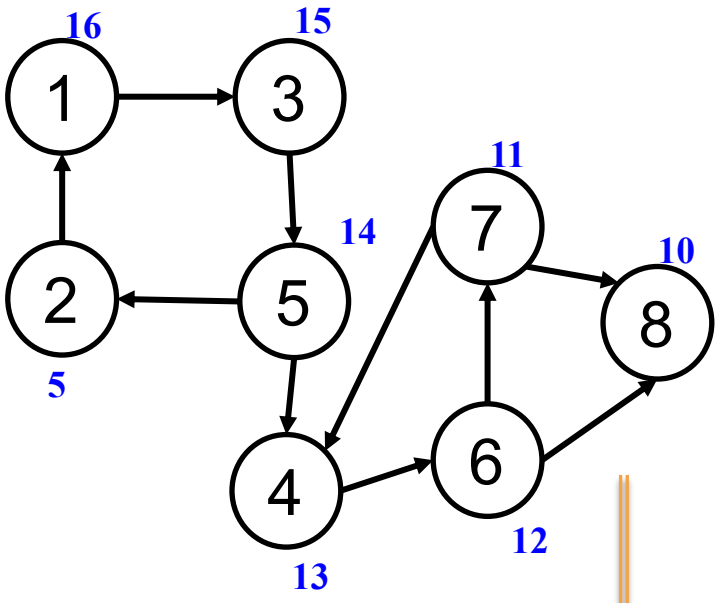
Maximal strongly connected subgraph

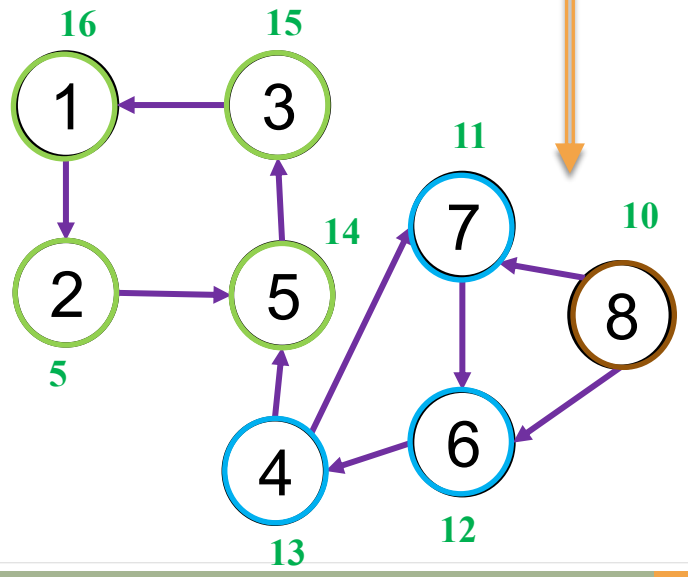
- A strongly connected component SCC in a directed graph $DG = \langle V, E \rangle$ is
  - a maximal subset of vertices such that:
  - for every pair of vertices x, y in C, there is a path between x and y in both direction.
  - Thus, starting at any vertex in a SCC, it is possible to reach every other vertex in C.
  - It is Maximal: You can't add more vertices to an SCC without losing strong connectivity.
- Finding SCC: Kosaraju's Algorithm (Two-pass DFS)
  - DFS ⟶ perform post-order traversal on DG
  - Transpose the graph ⟶ reverse all edges
  - DFS ⟶ perform DFS on the transposed graph in decreasing order of finish times
  - Each DFS result = **one SCC**
  - Time complexity $O(|V| + |E|)$

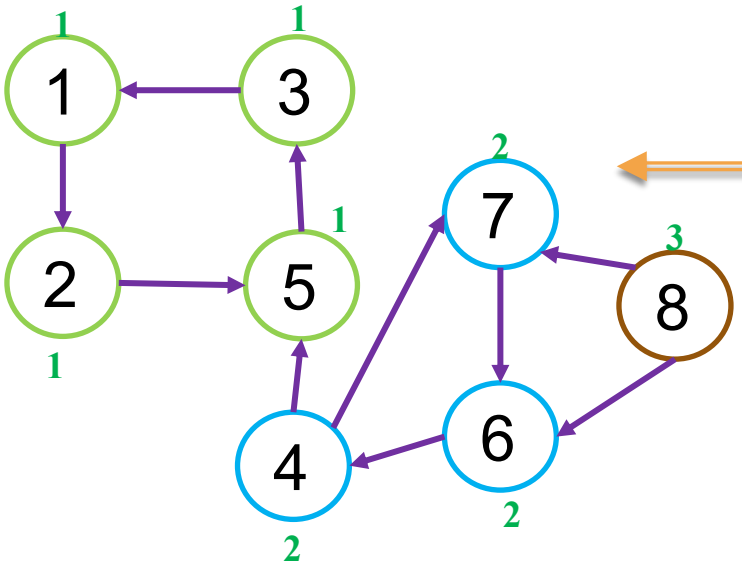


Example









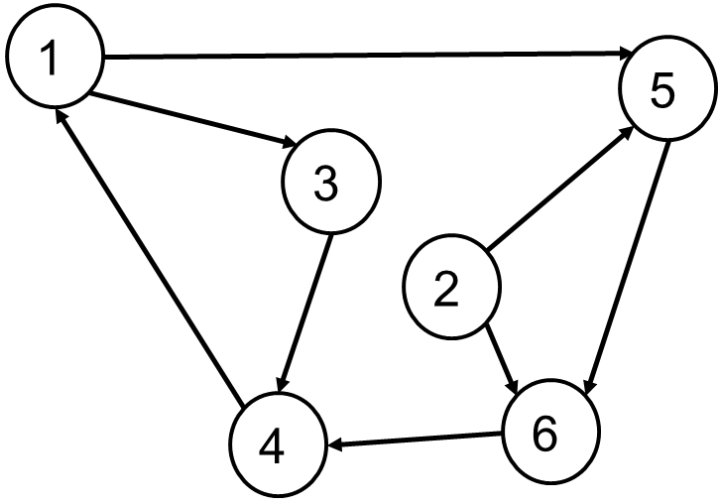
We have 3 SCCs

SCC1: 1, 2, 3 ,and 5

SCC2: 4, 6, 7

SCC3: 8

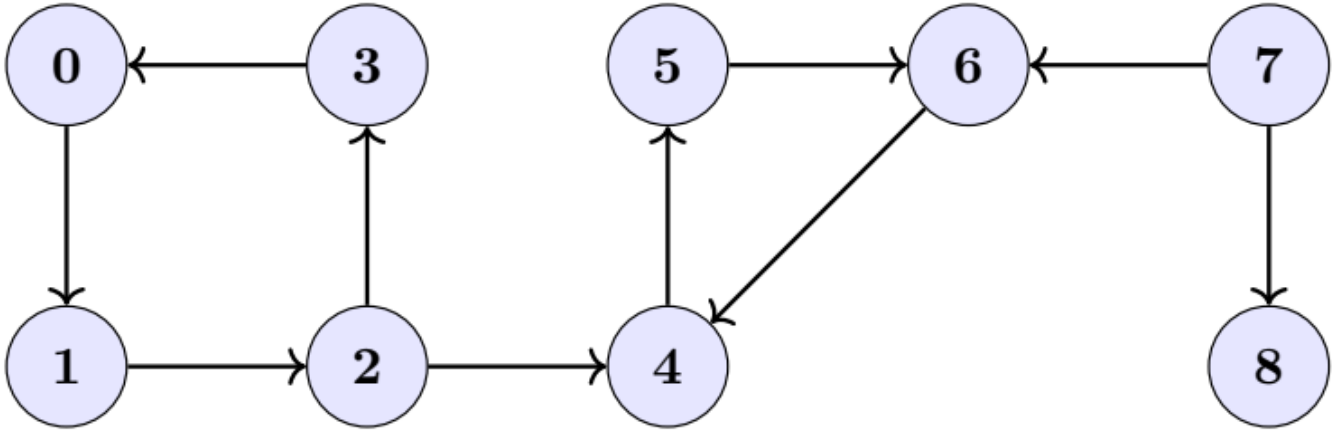
Activity: Run SCC Algorithm on



We have 2 SCCs

SCC1:

SCC2:



We have 4 SCCs

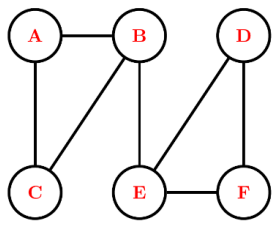
7

8

0-3-2-1

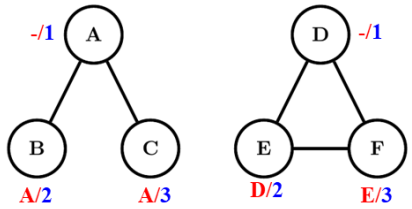
4-6-5

Cycles in an Undirected Graph



- A **cycle** in an undirected graph is a path that starts and ends at the same vertex
  - e.g. A ⟶B ⟶ C ⟶ A
- **How Cycles Form (Intuition)**
  - A cycle appears when you add an edge between **two vertices that were already connected** by some path.
- Cycle Detection in an Undirected Graph
- Method 1: DFS Cycle Detection : Cycle exists iff DFS finds a visited vertex that is not the predecessor of the current vertex.
  - In an undirected graph, the predecessor of a vertex v, denoted by $pred(u)$ is the immediate vertex from which a traversal reaches vertex v.
  - For every visited vertex v, if there is an adjacent vertex u such that u is already visited and u is not a predecessor of v, then there is a cycle in the graph.
- Method 2: Union-Find (Disjoint Set Union). Not covered

Step	1	2	3	4	5	6
Current node	A	B	C	D	E	F
Pred	-	A	A	-	D	E
Adjacent	B, C	A	A	E, F	D, F	E, D
Visited	A	A, B	A, B , C	A, B , C , D	A, B , C , D, E	A, B , C , D, E, F



D is an adjacent of F

D is visited

D is not a predecessor of F

Cycles in Undirected Graphs

```
// Before calling the algorithm, the following global variables
// are first set [in the general traversal algorithm]:
// startTime(v) = 0 for each vertex v ∈ V
// pred(v) = null for each vertex v ∈ V
// i = 1

cycleDetectionDFS(v)
 startTime(v) = i ; i++;
  for all vertices u adjacent to v
     if startTime(u) == 0
        pred(u) = v;
        cycleDetectionDFS(u);
     else if u ≠ pred(v)
           pred(u) = v;
           cycle detected;
```

```
int n ;
List<List<Integer>> graph;
// Build adjacency list
int[] startTime = new int[n];
int[] pred = new int[n];
int time = 1;

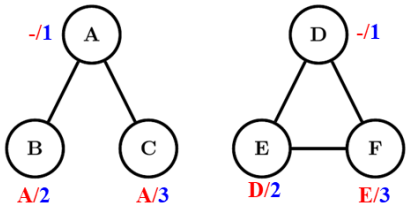
Arrays.fill(startTime,n);      // unvisited
Arrays.fill(pred, -1);


private boolean cycleDetectionDFS(int v) {
   startTime[v] = time++;

   for (Edge e : graph.get(v)) {
      int u = e.dist;  // neighbor

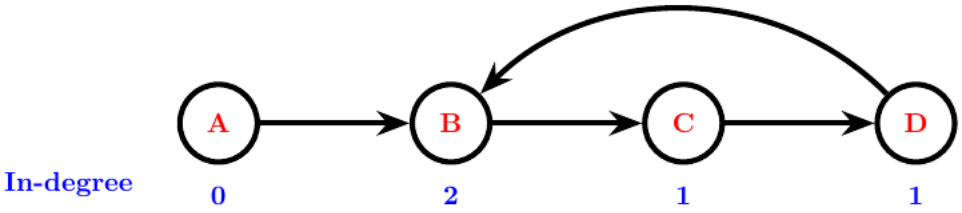
      if (startTime[u] == 0) {
         pred[u] = v;
         if (cycleDetectionDFS(u)) {
            return true;
         }
      } else if (u != pred[v]) {
         return true;
      }
   }
   return false;
}
```

Step	1	2	3	4	5	6
Current node	A	B	C	D	E	F
Pred	-	A	A	-	D	E
Adjacent	B, C	A	A	E, F	D, F	E, D
Visited	A	A, B	A, B , C	A, B , C , D	A, B , C , D, E	A, B , C , D, E, F



Cycles in a Directed Graph

- An easy way to detect the presence of cycles in a directed graph is to attempt a topological order traversal.
  - This algorithm visits all the vertices of a directed graph if the graph has no cycles.
- In the following graph, after A is visited and removed, all the remaining vertices have in-degree of one.
- Thus, a topological order traversal cannot complete. This is because of the presence of the cycle { B, C, D, B}.



What is the time complexity of this algorithm in O()? $O(|V| + |E|)$

Review Questions

- Every tree is a directed, acyclic graph (DAG), but there exist DAGs that are not trees.
  - How can we tell whether a given DAG is a tree?
  - Devise an algorithm to test whether a given DAG is a tree.

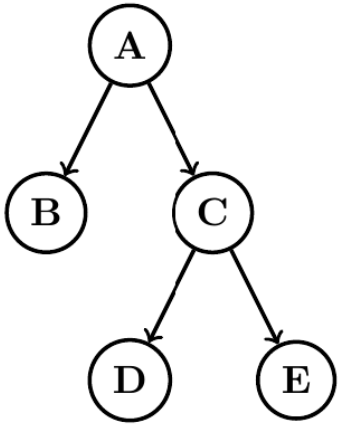


Figure 1: Tree

- Consider an acyclic, connected, undirected graph G that has n vertices. How many edges does G have?

- In general, an undirected graph contains one or more connected components.
  - Devise an algorithm that counts the number of connected components in a graph.
  - Devise an algorithm that labels the vertices of a graph in such a way that all the vertices in each connected component get the same label and vertices in different connected components get different labels.

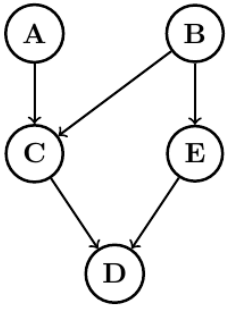


Figure 2: DAG but not a Tree

- Devise an algorithm that takes as input a graph, and a pair of vertices, v and w, and determines whether w is reachable from v.